

F1 STACKMIND Cyber Academy

System Documentation

Prepared by: Akira

Project type: Web-based Learning Management System

Project title: F1 STACKMIND Cyber Academy — A Cybersecurity Career-Training Platform for Mentors and Mentees

Date: September 2026

Table of Contents

1. Introduction
 2. Technical Background
 3. System Analysis
 4. System Design
 5. Implementation
 6. Testing and Evaluation
 7. Deployment
 8. User Manual
 9. Maintenance and Recommendations
 10. Conclusion
 11. References
-

Chapter 1 — Introduction

1.1 Project Background

F1 STACKMIND classes deliver a structured cybersecurity internship program. The program is built around weekly lessons, quizzes, and attendance tracking that a mentor supervises over an eight-week period.

Conventionally, such programs were administered manually: lessons were shared as files, attendance was recorded on paper or spreadsheet, and quiz results were tallied by hand. This created several problems: content was easy to distribute but difficult to version, quiz answers could not be protected against copying, attendance records

were scattered, and the mentor had no consolidated view of each mentee's progress.

The F1 STACKMIND Cyber Academy is a web application that digitizes the whole program. It provides a mentee portal for reading weekly lessons, answering anti-copy quizzes, marking attendance, and sending feedback, and a mentor portal for publishing content, building quizzes, monitoring mentee performance, and reviewing an audit trail.

1.2 Statement of the Problem

1. Weekly lesson materials were previously shared as loose files with no reliable version control or publication workflow.
2. Quiz tracking relied on manual checking; there was no protection against answer copying and no automatic scoring.
3. Attendance and performance records were not centralized, making it difficult for the mentor to evaluate mentees quickly.
4. There was no audit trail of who did what, and no channel for mentees to report issues or give feedback.
5. Accounts were created manually, with no verification that the person enrolling truly owned their email address.

1.3 Objectives

General objective — To develop F1 STACKMIND Cyber Academy, a secure web platform that delivers, tracks, and supervises an eight-week cybersecurity training program for mentees under a mentor.

Specific objectives:

1. Provide an OTP-verified account creation and login system for mentees and mentors.
2. Allow mentors to publish weekly lesson modules and build timed quizzes.
3. Let mentees read lessons, mark completion, answer quizzes with automatic scoring, and mark their weekly attendance.
4. Protect quiz integrity through anti-copy controls and server-side scoring.
5. Provide mentors a dashboard of mentee performance and a full audit log.
6. Give mentees a feedback-reporting channel and give mentors an inbox to resolve reports.
7. Enforce security through authentication, Row Level Security, rate limiting, input validation, and safe rendering.

1.4 Scope and Limitations

In scope:

- Two roles: mentor and mentee.
- Eight weekly modules and four server-scored quizzes, seeded into the database.
- Attendance marking per week; one record per mentee per week.
- Feedback reports submitted by mentees and triaged by the mentor.
- Audit logging of sensitive actions (signups, logins, attendance, quiz submissions, content changes, feedback actions).
- Deployment on Vercel with a Supabase backend.

Limitations:

- OTP email delivery depends on an SMTP provider being configured in Supabase; without SMTP, production code delivery is limited by the email rate limit.
- The in-memory rate limiter is scoped to a single server function instance; global enforcement would require a shared store such as Vercel KV or Upstash.
- Quiz anti-copy controls deter casual copying but are not a substitute for physical proctoring.
- The system is in English; no multi-language support yet.

1.5 Significance of the Study

- **Mentees** receive a structured, self-paced training environment with automatically scored quizzes and enforced attendance.
- **Mentor** gains a single dashboard for content publishing, quiz building, performance monitoring, feedback, and audit.
- **Faculty and Administration** can evaluate the system through this documentation and observe authenticated, audited program delivery.
- **Researchers / Developers** can study the applied security model (RLS, OTP, rate limiting, safe rendering) as a reference implementation.

1.6 Definition of Terms

- **Mentor** — an administrator/teacher account that publishes content, builds quizzes, and monitors mentees.
- **Mentee** — a learner account enrolled in the training program.
- **OTP** — One-Time Password; a six-digit code sent to the user's email to verify account ownership.
- **RLS** — Row Level Security; a database feature that restricts row access per user.
- **CIA triad** — Confidentiality, Integrity, Availability; the three core security goals.
- **Audit log** — a chronological record of sensitive actions.
- **CTF** — Capture The Flag; legal hacking practice competitions.

- **DMZ** — a perimeter network isolating internet-facing servers from internal systems.

Chapter 2 — Technical Background

2.1 Review of Related Concepts

Learning management. Existing platforms such as Moodle and Google Classroom share the same goals of distributing lessons and tracking learners. This system adds cybersecurity-specific needs that generic LMS platforms do not serve well: quiz answer protection, role-scoped auditing, and attendance tied to a weekly curriculum.

Modern web stacks. The application follows the React/Next.js App Router model, where pages are composed from React components and data is fetched server-side. State and routing live on the client; data access stays on the server so that secrets and row-level policies are never exposed to the browser.

Security in web applications. The OWASP Top 10 (injection, broken access control, XSS, broken authentication) guided the design. NIST SP 800-61 informed the incident-response secure-coding practices (least privilege, logging, input validation). Supabase provides the closest practical equivalent of a role-based access system on Postgres through Row Level Security.

2.2 Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 16 (App Router) + React 19	Page rendering, routing, server components
Styling	Tailwind CSS v4	Utility-first dark "terminal" theme
Language	TypeScript 5	Static typing across the codebase
Validation	Zod	Request schema validation on every API mutation
Backend / DB	Supabase (Postgres + Auth)	Database, authentication, Row Level Security
Icons	lucide-react	UI iconography
Hosting	Vercel	Production deployment (serverless functions)

Layer	Technology	Purpose
Version control	GitHub	Source control and deployment trigger

2.3 Development Tools

- VS Code editor; Git for version control; ESLint for code linting; `tsc` for type checking; `next build` for production builds; Playwright (Chromium) for automated walkthrough verification.

Chapter 3 — System Analysis

3.1 Analysis of the Existing (Manual) Process

Concern	Manual approach	Problem solved by this system
Lessons	Files shared one-way	Published modules with a page per week
Quizzes	Paper/verbal check	Timed, auto-scored, anti-copy quiz game
Attendance	Paper sign-in	One-tap weekly marking, mentor view of rates
Reports	None / informal	Feedback report form + mentor inbox
Oversight	None	Mentor dashboard, per-mentee performance, audit log
Accounts	Manually created	Self-service OTP-verified signup

3.2 Functional Requirements

1. The system shall allow a user to sign up with their email, verify it with a six-digit OTP, and log in with a password.
2. The system shall require a secret access code before creating a mentor account.
3. Mentors shall be able to create, edit, publish, and unpublish lesson modules and quizzes.
4. Mentees shall be able to list modules, read lesson content, and mark a module complete.
5. Mentees shall be able to take a timed quiz; the quiz shall auto-submit when the timer expires or after three tab-switch/copy violations.

6. The system shall score quizzes on the server and never expose correct answers to the browser.
7. Mentees shall be able to mark attendance for the current week (one record per week).
8. Mentees shall be able to submit feedback reports and view their status; mentors shall be able to update report status.
9. The system shall record an audit entry for every sensitive action.
10. The system shall restrict access by role at the route and API layers.

3.3 Non-Functional Requirements

- **Security** — HTTPS, HSTS, CSP, frame denial, HttpOnly session cookies, RLS on every table, server-side validation, generic error messages (no user enumeration).
- **Reliability** — production build passes type check, lint, and build; no client-side exceptions on the main flows.
- **Usability** — role-based navigation, clear status feedback, dark terminal theme consistent across pages.
- **Maintainability** — typed shared models, centralized client modules, documented schema and seed data.

3.4 Users and Roles

Role	Capabilities
Mentee	Dashboard with progress stats; read modules; mark completion; take quizzes; mark attendance; submit feedback; view own feedback history.
Mentor	Mentor overview; publish/manage modules; build/manage quizzes; view mentee performance; resolve feedback; read full audit log.
Unauthenticated	Landing page and the auth page only; everything else redirects to login.

3.5 System Flow (Use Case Summary)

1. A visitor opens the landing page and chooses to sign in.
2. To create an account: enter email (and, for a mentor, the access code) → an OTP is emailed → the six-digit code is verified → a password is set → the profile is auto-created.
3. A mentee with an active session reaches the Dashboard, reads modules, marks completion, takes quizzes, marks attendance, and sends feedback.
4. A mentor with an active session reaches the Mentor HQ, publishes content, builds quizzes, reviews mentee tables and audit logs, and resolves feedback.

5. Every sensitive action is recorded in the audit log with the acting user's email and IP.
 6. Logging out clears the session and returns to the auth page.
-

Chapter 4 — System Design

4.1 System Architecture

The application uses a server-rendered web architecture with a separation between browser, server, and database.

- The **browser** runs React pages and calls `/api/*` server routes with the session cookie.
- The **Vercel server runtime** runs Next.js pages and API routes. Sealed service-role credentials (`lib/supabase/admin.ts`) are held only on the server, never sent to the browser.
- **Supabase** provides Postgres with Row Level Security, the auth service (password hashing, OTP, session cookies), and the exposed REST API used by the server.

Three Supabase clients exist by design:

Client	Used where	Privilege
Browser client	Client components (AuthForm, feedback)	anon key; limited by RLS
Server cookie client	Server components, route handlers	user session; limited by RLS
Service-role admin client	Server-only (<code>lib/supabase/admin.ts</code>)	bypasses RLS; never shipped to browser

4.2 Database Design

Nine tables and one view are defined in `supabase/schema.sql` . All tables have Row Level Security enabled.

Table	Key Columns	Purpose
<code>profiles</code>	id, email, full_name, role, year_level	One row per auth user; auto-created at signup

Table	Key Columns	Purpose
<code>modules</code>	<code>week_no</code> , <code>title</code> , <code>description</code> , <code>content</code> , <code>video_url</code> , <code>published</code>	Weekly lesson content
<code>module_progress</code>	<code>user_id</code> , <code>module_id</code> , <code>completed</code> , <code>completed_at</code>	Per-mentee module completion
<code>attendances</code>	<code>user_id</code> , <code>week_no</code> , <code>status</code> , <code>note</code> , <code>date</code>	Weekly attendance (unique per user+week)
<code>quizzes</code>	<code>week_no</code> , <code>title</code> , <code>description</code> , <code>time_limit_sec</code> , <code>published</code>	Quiz definitions
<code>quiz_questions</code>	<code>quiz_id</code> , <code>question</code> , <code>options</code> , <code>correct_index</code> , <code>points</code> , <code>position</code>	Question bank (mentor-readable only)
<code>quiz_attempts</code>	<code>user_id</code> , <code>quiz_id</code> , <code>score</code> , <code>total</code> , <code>answers</code> , <code>started_at</code> , <code>finished_at</code>	Attempt history with server-side score
<code>activity_logs</code>	<code>user_id</code> , <code>email</code> , <code>action</code> , <code>details</code> , <code>ip</code> , <code>created_at</code>	Audit trail of sensitive actions
<code>feedback_reports</code>	<code>user_id</code> , <code>user_email</code> , <code>subject</code> , <code>message</code> , <code>category</code> , <code>status</code>	Mentee feedback/report channel
View <code>quiz_questions_public</code>	<code>id</code> , <code>quiz_id</code> , <code>question</code> , <code>options</code> , <code>position</code>	Sanitized question view (strips <code>correct_index</code>)

Table relationships: `profiles` extends `auth.users`; `module_progress` and `quiz_attempts` reference `profiles`; `attendances` and `feedback_reports` reference `profiles`; `quiz_questions` and `quiz_attempts` reference `quizzes` with cascade delete.

4.3 Role and Data Access Rules (RLS)

- Mentees read their own profile, progress, attendance, attempts, and feedback; they only see **published** modules and quizzes.
- Mentors read and manage everything; the `quiz_questions` table (which contains the correct answers) is **mentor-only** even from the database layer.
- A security-definer `is_mentor()` helper lets policies check roles without recursion.
- The audit log is mentor-readable only.

4.4 Security Design (Eight Layers)

1. **Transport security** — HTTPS with HSTS, a strict Content-Security-Policy, `X-Frame-Options: DENY`, and content-type options, set in `next.config.ts`.
2. **Authentication** — Supabase Auth; passwords are bcrypt-hashed; sessions are stored in HttpOnly, SameSite cookies via `@supabase/ssr`.

3. **Email OTP gate** — no usable account is created until a six-digit code sent to the enrolled email is verified through `/api/auth/verify`.
4. **Row Level Security** — every table has RLS; correct quiz answers are mentor-readable only and mentees consume a sanitized view.
5. **Role-based access** — route protection in `proxy.ts` plus server-side double checks in layouts and through `requireMentor()` on privileged APIs.
6. **Rate limiting** — an in-memory sliding window covers login, OTP send, and OTP verify; Supabase's email-send limit is also managed.
7. **Validation and CSRF safety** — Zod validates every mutation; SameSite cookies reduce CSRF; generic auth errors prevent user enumeration.
8. **Application guards** — every sensitive action is audit-logged; the quiz engine blocks copy/paste/view-source and detects DevTools and tab switches (three strikes = auto-submit); quiz answers are scored server-side; lesson content is rendered by a safe renderer that never uses `dangerouslySetInnerHTML`.

4.5 Anti-Copy Quiz Design

- Questions are fetched server-side with safe columns only; the browser never receives `correct_index`.
- The client sends the selected option indices; the score is computed on the server route `/api/quiz/submit` using the stored answers.
- The timer auto-submits; DevTools, tab-switch, and paste events increment a violations counter and auto-submit at the third strike.

4.6 Security-Header Configuration (example from `next.config.ts`)

Header	Value
Content-Security-Policy	default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; connect-src 'self' https://...supabase.co
Strict-Transport-Security	max-age=63072000; includeSubDomains; preload
X-Frame-Options	DENY
X-Content-Type-Options	nosniff
Referrer-Policy	strict-origin-when-cross-origin

Chapter 5 — Implementation

5.1 Development Environment

- Node.js 22, npm; Next.js 16.3.5; React 19.2.8; TypeScript 5; Tailwind CSS v4; Supabase JS client 2.116.0.
- Verification commands used during development: `npx tsc --noEmit`, `npm run lint`, `npm run build`.

5.2 Implemented Pages

Route	Audience	Function
<code>/</code>	Public	Landing page with the security-model explainer
<code>/auth</code>	Public	Login, OTP signup, OTP verify, password set
<code>/dashboard</code>	Mentee	Overview with progress stats and weekly greeting
<code>/dashboard/modules</code> , <code>/dashboard/modules/[id]</code>	Mentee	Module list and lesson reader with "mark complete"
<code>/dashboard/quiz</code> , <code>/dashboard/quiz/[id]</code>	Mentee	Quiz list and timed quiz player
<code>/dashboard/attendance</code>	Mentee	Weekly attendance marking with history
<code>/dashboard/feedback</code>	Mentee	Feedback submission and report history
<code>/mentor</code>	Mentor	Mentor overview with program statistics
<code>/mentor/modules</code>	Mentor	Module manager (create/edit/publish)
<code>/mentor/quiz</code>	Mentor	Quiz builder with live question management
<code>/mentor/mentees</code>	Mentor	Per-mentee performance table
<code>/mentor/feedback</code>	Mentor	Feedback inbox with status updates
<code>/mentor/logs</code>	Mentor	Audit log viewer with filters

5.3 Implemented API Routes

Route	Method	Function
<code>/api/auth/otp</code>	POST	Send a one-time password
<code>/api/auth/verify</code>	POST	Verify OTP and create session
<code>/api/auth/login</code>	POST	Password login
<code>/api/auth/logout</code>	POST	End the session
<code>/api/auth/session</code>	GET	Read the current session
<code>/api/attendance/mark</code>	POST	Upsert weekly attendance
<code>/api/quiz/submit</code>	POST	Server-side scoring of a quiz attempt
<code>/api/mentee/module-progress</code>	POST	Toggle module completion
<code>/api/mentor/modules</code> , <code>/api/mentor/modules/[id]</code>	GET/POST/PATCH/DELETE	Module CRUD
<code>/api/mentor/quiz</code> , <code>/api/mentor/quiz/[id]</code>	GET/POST/PATCH/DELETE	Quiz and question CRUD
<code>/api/mentor/mentees</code>	GET	Mentee performance data
<code>/api/mentor/logs</code>	GET	Filtered audit log
<code>/api/feedback</code>	GET/POST	List and submit feedback
<code>/api/feedback/[id]</code>	PATCH	Update feedback status (mentor)

5.4 Content Implementation

- `supabase/schema.sql` creates tables, triggers, policies, and the safe view.
- `supabase/seed.sql` loads eight weekly modules (Weeks 1–8: Introduction, Networking, Linux, Cryptography, Web Security, Network Defense, Incident Response, Ethical Hacking) and four quizzes of five questions each; it is re-runnable (upsert by `week_no`).
- Module content is authored in a restricted Markdown subset and rendered by `MarkdownLite`, which supports headings, lists, bold, and inline code without any unsafe HTML.

5.5 Known Implementation Notes

- The quiz page fetches safe columns server-side with the service-role client so RLS (which keeps `quiz_questions` mentor-only) can never leak answers or return zero rows to mentees.
 - The OTP rate limit was raised to 10 sends per 10 minutes after free-tier email limits were observed.
-

Chapter 6 — Testing and Evaluation

6.1 Verification Commands

Check	Command	Result
Type safety	<code>npx tsc --noEmit</code>	Passed — no type errors
Code lint	<code>npm run lint</code>	Passed — zero errors
Production build	<code>npm run build</code>	Passed — all 18 routes generated
Deployment	Vercel auto-deploy on push	Succeeded

6.2 Database Testing

Test	Procedure	Result
Schema application	Ran <code>schema.sql</code> in Supabase SQL editor	Passed — all tables, policies, view, triggers created
Seed data	Ran <code>seed.sql</code>	Passed — 8 modules and 4 quizzes loaded
Re-runnable seed	Re-ran modules-upsert	Passed — no duplicate rows

6.3 Functional Test Cases

ID	Test Case	Expected Result	Actual Result
F1	Mentor signs up with the access code	Mentor role created	Passed

ID	Test Case	Expected Result	Actual Result
F2	Mentee accounts created via admin API	Six mentees with names and year level	Passed
F3	Mentee logs in with email + password	Session created, dashboard greets by first name	Passed
F4	Mentee opens a module	Lesson content renders correctly	Passed
F5	Mentee takes the Week 2 quiz	Anti-copy rules apply; submission scores on server	Passed
F6	Mentee marks attendance	One record for the week is saved	Passed
F7	Mentee submits feedback	Report appears in the mentor inbox	Passed
F8	Mentor updates feedback status	Status pill updates for the mentee	Passed
F9	Non-mentor calls a mentor API	403 Forbidden	Passed
F10	Unauthenticated visits /dashboard	Redirects to /auth	Passed

Note: quiz answers never reach the browser; the browser receives only question text and option labels, and the score is returned from the server after submission.

6.4 Site Availability

- The application is live at `https://cyber-academy-sooty.vercel.app` with Supabase project `lsetomnqaydzrgcyngtn`.
- Test accounts used during evaluation: `mentor@cyber.academy`, `mentee@cyber.academy`, and `mentee1..6@cyber.academy` (documented in the user manual).

Chapter 7 — Deployment

7.1 Environment Variables

Variable	Purpose
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Supabase project URL

Variable	Purpose
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public anon key (browser-safe)
SUPABASE_SERVICE_ROLE_KEY	Service-role key (server-only)
MENTOR_ACCESS_CODE	Secret code required for mentor signup

7.2 Deploy Steps

1. Create a Supabase project; run `schema.sql` and `seed.sql` in the SQL Editor; enable the Email provider with OTP; add the Vercel domain to the allowed URLs.
2. Push the code to GitHub and import the repository in Vercel.
3. Set the four environment variables in Vercel (Production and Preview).
4. Deploy; Vercel rebuilds on every push to `main`.

7.3 Frontend/Backend Budgeting Note

- Sealed admin client (`lib/supabase/admin.ts`) uses the service-role key and is only imported by server modules (`server-only`), so it is never bundled for the browser.
 - The CSP must permit Next.js server-rendered scripts; the header includes `unsafe-inline / unsafe-eval` for the framework runtime while retained for other source directives.
-

Chapter 8 — User Manual

8.1 Getting Started

Open `https://cyber-academy-sooty.vercel.app`. Choose **Sign in** from the landing page.

8.2 Creating an Account

1. Enter your email address.
2. For mentor signup, enter the secret access code provided by the F1 STACKMIND program.
3. Enter the six-digit code emailed to you.
4. Set a password that is at least eight characters and contains an uppercase letter, a lowercase letter, a number, and a special character.

8.3 Mentee Guide

- **Overview** — view weekly attendance, completed modules, quizzes taken, and best score.
- **Modules** — open a week's lesson, read the content, then press **Mark as complete**.
- **Quizzes** — select a quiz, press **Start challenge**, answer each question, then **Submit**. Do not switch tabs or open Developer Tools: three violations auto-submit the quiz. If time runs out, the quiz submits automatically.
- **Attendance** — press **Mark present** for the current week.
- **Feedback** — use the form to report issues or suggestions; track the status (new / in review / resolved) on the same page.
- **Signing out** — use the power icon in the navigation bar.

8.4 Mentor Guide

- **Overview** — program statistics at a glance.
- **Modules** — create a module, write lesson content (MarkdownLite format), and publish it so mentees can see it.
- **Quizzes** — add a quiz with a title, time limit, and four-option questions with a designated correct index; publish when ready.
- **Mentees** — review each mentee's attendance, quiz best/avg scores, and completed modules.
- **Feedback** — read incoming reports and move them through **In review** and **Resolved**.
- **Audit Logs** — filter events by action and email to investigate any action taken on the platform.

8.5 Test Accounts

Account	Purpose
mentor@cyber.academy / Mentor#2026!	Mentor demonstration account
mentee@cyber.academy / Mentee#2026!	Mentee demonstration account
mentee1@cyber.academy ... mentee6@cyber.academy / Mentee#2026!	Six mentee records (3rd Year) including D-Renz Sabordo, James Marco, Jeremie, Jeson Gallego, Marc Cedric Echalar, Michael Ebalan Elambre

Chapter 9 — Maintenance and Recommendations

9.1 Maintenance

- Source, schema, and seed are version-controlled in GitHub (`Universal43A2z/cyber-academy`).
- Content changes are made through the mentor UI or by updating the seed and re-running the upsert.
- Feature changes follow the same pipeline: verify with `tsc` , lint, and build, then push to `main` .

9.2 Recommendations

1. Configure an SMTP provider (e.g., Resend) in Supabase so OTP emails are delivered reliably to mentees.
2. Replace the in-memory rate limiter with a shared store (Vercel KV / Upstash) for enforcement across all server instances.
3. Add Google Sign-In as a convenience login option.
4. Enroll real mentees flagged at risk of academic dishonesty through stricter proctoring (e.g., camera-based).
5. Add report generation (PDF) per mentee for teacher and dean review.
6. Move module content to a document-based editor instead of raw Markdown for non-technical mentors.

Chapter 10 — Conclusion

F1 STACKMIND Cyber Academy meets its stated objectives. It delivers the eight-week curriculum through a mentee portal, protects quiz integrity with anti-copy controls and server-side scoring, centralizes attendance and performance data for mentors, verifies email ownership with OTP, and maintains a complete audit trail for every sensitive action. The layered security model — transport hardening, authentication, OTP gating, Row Level Security, role checks, rate limiting, validation, and application guards — keeps correct quiz answers, session data, and service credentials out of the browser and out of the hands of unauthorized users.

The system was verified through type checks, linting, a successful production build, database seeding, functional test cases, and a live deployment. It is ready for demonstration to mentors, faculty, and the dean, and can be extended with SMTP

mail, shared rate limiting, login conveniences, and per-mentee reporting as listed in the recommendations.

Chapter 11 — References

1. Next.js Documentation. App Router, Server Components, Route Handlers.
<https://nextjs.org/docs>
2. Supabase Documentation. Authentication, Row Level Security, Postgres.
<https://supabase.com/docs>
3. OWASP Foundation. OWASP Top 10 (2021). <https://owasp.org/Top10/>
4. NIST. SP 800-61 Rev. 2, Computer Security Incident Handling Guide.
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>
5. React Documentation. <https://react.dev>
6. Tailwind CSS Documentation. <https://tailwindcss.com/docs>
7. Zod Documentation. Schema validation. <https://zod.dev>
8. Vercel Documentation. Serverless Functions, Environment Variables.
<https://vercel.com/docs>